Assignment No: 4(B)

```
#include <stdio.h>

#define N 3

__global__ void matrixMultiply(int *A, int *B, int *C)
{
    int row = threadIdx.y;
    int col = threadIdx.x;

    int sum = 0;

    for(int k = 0; k < N; k++)
    {
        sum += A[row * N + k] * B[k * N + col];
    }

    C[row * N + col] = sum;
}

int main()
{
    int A[N][N] = {
        {1,2,3},
        {4,5,6},
        {7,8,9}
    };

    int B[N][N] = {
        {1,0,0},
        {0,1,0},
        {0,0,1}
    };

    int C[N][N];

    int *d_A, *d_B, *d_C;
    int size = N*N*sizeof(int);

    cudaMalloc(&d_A,size);
    cudaMalloc(&d_B,size);
    cudaMalloc(&d_C,size);

    cudaMemcpy(d_A,A,size,cudaMemcpyHostToDevice);
    cudaMemcpy(d_B,B,size,cudaMemcpyHostToDevice);
```

```
    dim3 threads(N,N);

    matrixMultiply<<<1,threads>>>(d_A,d_B,d_C);

    cudaMemcpy(C,d_C,size,cudaMemcpyDeviceToHost);

    printf("Result Matrix:\n");

    for(int i=0;i<N;i++)
    {
        for(int j=0;j<N;j++)
        {
            printf("%d ",C[i][j]);
        }
        printf("\n");
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

Output:

Running in FUNCTIONAL mode...
Compiling...
Executing...
Result Matrix:
1 2 3
4 5 6
7 8 9
Exit status: 0